

Contents

1	認証・認可設計書 (メイン)	1
1.1	1. 文書概要	1
1.1.1	1.1 目的	1
1.1.2	1.2 対象範囲	1
1.1.3	1.3 版数	1
1.1.4	1.4 関連ドキュメント	2
1.2	2. 認証方式	2
1.3	3. 認証フロー	2
1.3.1	3.1 ログイン (AUTH-FLOW-001)	3
1.3.2	3.2 ログアウト	3
1.3.3	3.3 パスワード再設定 (AUTH-FLOW-003)	3
1.3.4	3.4 招待受諾 (AUTH-FLOW-004)	3
1.3.5	3.5 規約再同意 (AUTH-FLOW-005)	3
1.3.6	3.6 メール確認 (AUTH-FLOW-006)	3
1.3.7	3.7 エンドユーザー再入室 (AUTH-FLOW-007)	4
1.3.8	3.8 強制ログアウト (AUTH-FLOW-008)	4
1.4	4. セッション・トークン設計	4
1.4.1	4.1 トークン用途別仕様 (§ 10.1.3 から移管)	4
1.4.2	4.2 Cookie 属性 (§ 10.1.4)	4
1.5	5. 認可モデル	4
1.5.1	5.1 認可方式	4
1.5.2	5.2 オーナー境界判定式 (共有概念正本)	5
1.5.3	5.3 ユーザー種別の DB 値 (§ 3.1 から移管)	5
1.5.4	5.4 オーナー専有機能	5
1.5.5	5.5 プラン制限との関係	5
1.6	6. 認可判定ルール	5
1.6.1	6.1 Principal 構築フロー (§ 3.3.1 から移管)	5
1.6.2	6.2 認可判定の順序	6
1.6.3	6.3 critical 通知の宛先解決 (共有概念正本)	7
1.6.4	6.4 重要操作の再認証 (FR-005)	7
1.6.5	6.5 オーナー保護 (FR-333)/ 自己操作不可 (FR-334)/ 単一オーナー所属 (FR-335)	7
1.7	7. アカウント状態	7
1.7.1	7.1 サスペンション中の認可ゲート (§ 6.2.7 から移管)	8
1.7.2	7.2 退会・解約フロー (§ 6.2.8 から移管)	8
1.7.3	7.3 規約改定フロー (§ 6.2.9 から移管)	8
1.8	8. 認証・認可エラー	9
1.9	9. 未決事項	9
1.9.1	9.1 MVP リスク受容 (§ 10.1.5)	10

1 認証・認可設計書 (メイン)

1.1 1. 文書概要

1.1.1 1.1 目的

メインシステムの認証(ログイン/ログアウト/パスワード再設定/招待受諾/規約再同意/メール確認)、セッション・トークン、認可モデル(オーナー境界判定/Principal 構築)、認可判定ルール、アカウント状態、認証・認可エラーを一元化する。

1.1.2 1.2 対象範囲

- 対象: 利用者(オーナー/メンバー/エンドユーザー)の認証・認可全般
- 対象外: 運営者の認証・認可(MFA/IP allowlist/4-eyes は [02_運営者システム/個別設計書群/09_認証認可設計書.md](#) 正本)、エンドユーザーのウィジェット内認証詳細は API 設計書(03)に委譲

1.1.3 1.3 版数

項目	値
版数	1.0
更新日	2026-05-17

1.1.4 1.4 関連ドキュメント

ドキュメント名	役割	参照先
索引	11ドキュメント体系の俯瞰	00_索引.md
権限設計書	3 ロール定義 / 画面別権限	05_権限設計書.md
セキュリティ設計書	Argon2id / HMAC / 監査 / Cookie 属性	10_セキュリティ設計書.md
API 設計書	認証 API 仕様	03_API 設計書.md
エラー設計書	認証・認可エラー	06_エラー設計書.md
メッセージ一覧	通知重要度 / critical メール強制送信	07_メッセージ一覧.md
テーブル定義書	accounts / sessions / terms_agreements	04_テーブル定義書.md
課金・請求設計書	サスペンション時のアクセス制限	11_課金請求設計書.md
運営者側 認証・認可設計書	運営者ログイン向け IP allowlist(operator_ip_allowlist、独立スキーマ)	../02_運営者システム/個別設計書群/09_認証認可設計書.md

1.2 2. 認証方式

認証方式	利用有無	概要	備考
メールアドレス / パスワード	○	メイン認証方式 (Argon2id m=64MB, t=3, p=4, salt=16B)	パスワード要件は § 4
Google ログイン	×	MVP 範囲外	Future
SSO / SAML	×	MVP 範囲外	Future
OAuth / OIDC	×	MVP 範囲外	Future
MFA(利用者側 admin)	×(MVP 受容)	MVP では admin にも MFA 必須化しない	§ 10 でリスク受容、Future 移行
招待トークン	○	メンバー招待時に短期トークンで初回パスワード設定	§ 3.4
パスワード再設定トークン	○	メール経由の短期トークン (有効期限 1 時間)	§ 3.3
規約再同意	○	規約改定時に割込みフロー (SCR-025)	§ 3.5
メール確認	○	新規登録後の本人確認 (SCR-023)	§ 3.6
ウィジェット再入室トークン	○	エンドユーザー再入室用 (有効期限 30 日)	§ 3.7
Turnstile(reCAPTCHA 相当)	○	アカウント単位 3 回失敗 または 同一 IP 1 分 10 回失敗で発動 (FR-177)	§ 10.1

1.3 3. 認証フロー

フロー ID	フロー名	概要	関連画面 ID	関連 API ID
AUTH-FLOW-001	ログイン	メール + パスワード	SCR-001	POST /auth/login
AUTH-FLOW-002	ログアウト	セッション削除	(グローバル)	POST /auth/logout
AUTH-FLOW-003	パスワード再設定	メール経由トークン	SCR-003	POST /auth/password-reset-request, POST /auth/password/reset
AUTH-FLOW-004	招待受諾	招待トークン + 初回パスワード	SCR-017-M1 → SCR-001 派生	POST /invitations/:token/accept

フロー ID	フロー名	概要	関連画面 ID	関連 API ID
AUTH-FLOW-005	規約再同意	規約改定時の割込み	SCR-025	POST /terms/agree
AUTH-FLOW-006	メール確認	新規登録後のメール確認	SCR-023	POST /auth/email-verifications/{token}
AUTH-FLOW-007	エンドユーザー再入室	アクセストークン経由	SCR-027	GET /widget/sessions/:token
AUTH-FLOW-008	強制ログアウト (連携 IF #2)	運営者起動	(グローバル)	(IF #2 受信)

1.3.1 3.1 ログイン (AUTH-FLOW-001)

1. メール+パスワード入力 → Turnstile 検証 (FR-177 発動時)
2. Argon2id 検証 → **sessions** レコード作成 + Cookie 発行
3. 規約再同意状態確認 → 未同意なら SCR-025 へリダイレクト
4. 契約状態確認 (**contract_owners.contract_status**) → **suspended** で許可機能外なら 403、**deleted** なら 403
5. SCR-015 ダッシュボードへ

エラー: 401 **INVALID_CREDENTIALS**(E-AUTH-CREDENTIAL)、423 **LOCKED_OUT**(E-AUTH-LOCKED)、403 **CONTRACT_SUSPENDED**

1.3.2 3.2 ログアウト

セッション失効 + Cookie 削除。SCR-001 へ。

1.3.3 3.3 パスワード再設定 (AUTH-FLOW-003)

1. SCR-003 段階 1: メール入力 → **POST /auth/password-reset-request**(列挙攻撃対策で存在有無を返さない)
2. メール送信 (**access_tokens.purpose='password_reset'**、HMAC-SHA256、有効期限 1 時間、1 回限り、FR-006)
3. SCR-003 段階 2: トークン検証 + 新パスワード入力 → **POST /auth/password/reset**
4. 全セッション失効 → SCR-001 へ

1.3.4 3.4 招待受諾 (AUTH-FLOW-004)

1. プロジェクト WS の SCR-017-M1 から **POST /projects/{id}/members** → 招待メール送信 (**access_tokens.purpose='activation'**、有効期限 7 日、FR-016)
2. メール内リンク → 初回パスワード設定画面 → **POST /invitations/:token/accept**
3. **users.status='active'** + 当該プロジェクト分の **project_users** 行を反映 (他プロジェクトに参加させる場合は、対象プロジェクトの WS で個別に招待操作を行い、名前解決により既存ユーザーに行が追加される)
4. SCR-001 へ

1.3.5 3.5 規約再同意 (AUTH-FLOW-005)

1. ログイン直後に未同意があれば SCR-025 強制割込み
2. 規約 + プライバシーポリシー同意 (両必須) → **POST /terms/agree**
3. **terms_agreements** レコード作成
4. 同意期限超過後の機能制限ガード: 課金画面操作 + 新規プロジェクト作成は不可、FAQ 編集とウィジェット稼働は継続

1.3.6 3.6 メール確認 (AUTH-FLOW-006)

1. SCR-002 新規登録 → 確認メール送信 (**access_tokens.purpose='email_verify'**、有効期限 24 時間)

2. メール内リンク → POST /auth/email-verifications/{token}
3. users.email_verified_at 設定 → SCR-001 へ

1.3.7 3.7 エンドユーザー再入室 (AUTH-FLOW-007)

1. 案件メール内リンク → SCR-027
2. トークン形式: 不透明文字列 + HMAC-SHA256(オーナー派生、access_tokens.purpose='reentry'、有効期限 30 日)
3. トークン検証 → inquiry_id 復号 → SCR-013 個別チャット部屋へ
4. 期限切れ時は新リンクをメールで再送可能

1.3.8 3.8 強制ログアウト (AUTH-FLOW-008)

連携 IF #2 受信時、operator_manual / tos_violation の場合は 5 秒以内 に全セッション無効化 (FR-022)。payment_failure_grace_expired 等は既存セッション継続。

1.4 4. セッション・トークン設計

項目	内容
セッション方式	Cookie + DB セッション (sessions テーブル)
無操作タイムアウト	30 分 (FR-008(a))
絶対タイムアウト	12 時間 (FR-008(b))
再認証 (FR-005)	重要操作前に要求 (パスワード変更 / 退会 / 課金情報変更 / 管理者ユーザー登録・停止・削除 / 月次予算上限変更)。有効期間 = 当該操作 1 回のみかつ 15 分以内
Cookie 属性	Secure; HttpOnly; SameSite=Lax; Path=/、Domain 明示せず
CSRF	Double Submit Cookie + X-CSRF-Token ヘッダ突合
パスワードハッシュ	Argon2id(admin/end_user: m=64MB, t=3, p=4, salt=16B)
HMAC	HMAC-SHA256(各種トークン署名)
ログイン失敗ロックアウト	5 回失敗 → (IP × user_id) を 15 分ロック (FR-007 / FR-177)
強制ログアウト	連携 IF #2 受信時、アクセストークン全失効 (5 秒以内)
アクティブセッション一覧	SCR-001 ログイン後にヘッダ右上アカウントメニュー → 「アカウント設定」 → SCR-028 §(2) アクティブセッション から表示可能 (FR-332)
複数デバイス同時ログイン	可能 (FR-332)

1.4.1 4.1 トークン用途別仕様 (§10.1.3 から移管)

用途	鍵	トークン形式	有効期限
登録完了トークン	オーナー派生	不透明文字列 + HMAC	7 日
再入室トークン	オーナー派生	不透明文字列 + HMAC + inquiry_id 内包	30 日
削除確認トークン	オーナー派生	不透明文字列 + HMAC	24 時間
パスワードリセットトークン	グローバル派生	不透明文字列 + HMAC	1 時間
API キー (ウィジェット公開キー)	グローバル派生	pk_live_<32-char base62>	7 / 30 / 90 / 180 / 365 日 選択

1.4.2 4.2 Cookie 属性 (§10.1.4)

Cookie 種別	属性
メインシステム セッション Cookie	Secure; HttpOnly; SameSite=Lax; Path=/、Domain 明示せず
CSRF Cookie	Secure; SameSite=Lax; Path=/(HttpOnly なし、JS から読取可)

1.5 5. 認可モデル

1.5.1 5.1 認可方式

操作者種別 (管理者 = users 行存在 / エンドユーザー = end_users 行存在 / 運営者 = 別 D1) + プロジェクト別ロール project_users.role = admin / member + オーナー判定 = contract_owners 行存在

1.5.2 5.2 オーナー境界判定式 (共有概念正本)

すべての操作で以下の判定式を必須:

`principal.contractOwnerUserId === target.contract_owner_user_id`

- `principal` = 操作するユーザー (セッションから取得した Principal)
- `target` = 操作対象リソース (テーブルの `contract_owner_user_id` カラム)
- 一致しない場合は **404 偽装** (403 を返すと相手契約の存在が漏れる、エラー設計書 06 §5.3)

1.5.3 5.3 ユーザー種別の DB 値 (§3.1 から移管)

ユーザー種別	DB 値	概要
オーナー	users 行存在 AND contract_owners.user_id = users.id 行が存在	契約あたり1オーナー固定 (contract_owners.user_id PRIMARY KEY で DB レベル担保)。全権 + オーナー専有機能
プロジェクト管理者 / メンバー	users 行存在 AND contract_owners 行を持たない AND project_users 行を保持	プロジェクト別ロール (project_users.role = admin/member) で操作範囲が決まる
エンドユーザー	end_users 行存在 (プロジェクト単位)	自分の問い合わせ ID に紐づくチャットのみ。users 行は持たない
サービス運営者	別 D1(運営者システム service_operators テーブル)	本 SaaS 提供者 (運営者側、メイン側 D1 には格納しない)

主体テーブルの使い分け: 管理者は `users` 行 (全行が管理者)、エンドユーザーは `end_users` 行 (プロジェクト単位)。両者は同一テーブルを共有せず、認可フローでも別経路で Principal を構築する。オーナー判定: `contract_owners.user_id = users.id` 行存在 (`is_owner` カラムや `owner_account_id` 自己参照は持たない) `contract_owners.contract_status` 値域: `active` / `suspended` / `deleted_pending` / `deleted` の 4 値固定

1.5.4 5.4 オーナー専有機能

プロジェクト別ロール (`admin` / `member`) では付与不可な機能:

- プロジェクトの作成・編集・削除 (プロジェクト設定全般: 名称・許可ドメイン・ウィジェット ローター ション 等)
- 課金 (SCR-015 の月次予算変更 / 支払い方法変更)
- 退会 (SCR-024)
- 規約再同意 (SCR-025 の同意 / 不同意)
- メンバーアカウント論理削除 (`DELETE /projects/{id}/members/{userId}`)—— オーナー操作時は全メンバー削除可、プロジェクト管理者 (オーナー以外) 操作時は当該プロジェクトの `member` のみ削除可 (他 `admin` は `member` 在籍中削除不可)

UI 上は非表示 + API は 403(`E-AUTHZ-OWNER-ONLY`)。

1.5.5 5.5 プラン制限との関係

MVP は単一プラン。契約別利用上書き (`owner_quota_overrides` の `resource_kind` で統一実装) は認可判定の **次段** で適用される (まず認可 → 通過したら上限チェック → 通過したら処理実行)。詳細は [11_課金請求設計書.md §3](#) 参照。

1.6 6. 認可判定ルール

1.6.1 6.1 Principal 構築フロー (§3.3.1 から移管)

1.6.1.1 Principal オブジェクト構造

```
Principal {
  userId:          string
  contractOwnerUserId: string // 本人がオーナーなら自分の userId、メン/
  actorRole:       'admin' | 'end_user' | 'service_operator' // 操作者種別。管理者は users
  isOwner:         boolean // contract_owners 行が存在するかの boo
  projectRoles:    Map<string /*projectId*/, 'admin' | 'member'> // オーナーも project_U
}
```

1.6.1.2 Principal 構築クエリ (参考)

-- 管理者 (*users*) 向け *Principal* 構築

```
SELECT u.id, u.status, u.valid,
       (co.user_id IS NOT NULL) AS is_owner,
       COALESCE(co.user_id, pu.contract_owner_user_id) AS contract_owner_user_id
FROM users u
LEFT JOIN contract_owners co ON co.user_id = u.id AND co.valid = 1
LEFT JOIN project_users pu ON pu.user_id = u.id AND pu.valid = 1
WHERE u.id = ?1 AND u.valid = 1
LIMIT 1;
```

-- エンドユーザー (*end_users*) 向け *Principal* 構築 (プロジェクトスコープ)

```
SELECT eu.id, eu.project_id, eu.contract_owner_user_id, eu.verified, eu.valid
FROM end_users eu
WHERE eu.id = ?1 AND eu.valid = 1
LIMIT 1;
```

projectRoles は別途 SELECT project_id, role FROM project_users WHERE user_id=? AND valid=1 で取得する。

1.6.1.3 認可ミドルウェア実装

- ・オーナーはプロジェクト作成時に自動で project_users(role='admin', valid=1) 行を保持するため、users と contract_owners / project_users を結合する処理はオーナー / メンバー共通。ただし認可上は isOwner=true 判定 (= contract_owners 行存在) が成立した時点で projectRoles の中身を参照せず bypass で全プロジェクト許可する(認可優先順位)
- ・projectRoles は通知宛先解決と画面表示の事実情報として参照される(空 Map ではなく、オーナー自身の admin 行も含む)
- ・取得クエリには WHERE project_users.valid=1 フィルタを必須付与(論理削除済み行は除外)
- ・認可判定ヘルパ:
 - requireOwner(): principal.isOwner でない場合 403(E-AUTHZ-OWNER-ONLY)
 - requireProjectRole(projectId, minRole: 'admin' | 'member'): principal.isOwner ならば常時許可。それ以外は principal.projectRoles.get(projectId) が未定義なら 404 偽装(E-AUTHZ-PROJECT-DENIED)、minRole='admin' のときロールが admin でなければ 403(E-AUTHZ-FORBIDDEN)
- ・Principal: KV キャッシュ (TTL 60 秒) 経由で読み取り
- ・ロール変更時の即時反映よりも認可レイテンシ最適化を優先 (FR-338)

1.6.2 6.2 認可判定の順序

1. セッション検証 (sessions テーブル参照、無操作 30 分 / 絶対 12 時間)—— 失敗 → 401
2. ユーザー論理削除確認 (users.valid=1)—— valid=0 なら 401(セッション失効扱い、ログイン画面ヘリダイレクト)
3. 規約再同意状態確認 (terms_agreements 最新版)—— 未同意 → SCR-025 ヘリダイレクト
4. 契約状態確認 (contract_owners.contract_status、所属契約)—— suspended で許可機能外なら 403、deleted_pending / deleted なら 403
5. アカウント状態確認 (users.status='active')—— pending_activation / pending_verification なら 403
6. オーナー境界判定 (principal.contractOwnerUserId === target.contract_owner_user_id)—— 違反 → 404 偽装
7. プロジェクトロール判定 (requireProjectRole):
 - ・ principal.isOwner=true のとき自契約配下の全プロジェクトに無条件アクセス可 (bypass)
 - ・ メンバーは対象 project_id が principal.projectRoles に含まれ、必要ロール以上を満たすこと
 - ・ 違反 → 404 偽装 (E-AUTHZ-PROJECT-DENIED) / ロール不足 → 403 (E-AUTHZ-FORBIDDEN)
8. オーナー専有機能判定 (requireOwner、contract_owners 行を持たないユーザーがプロジェクト作成 / 課金 / 退会 / 規約再同意 等にアクセス)—— 違反 → 403 (E-AUTHZ-OWNER-ONLY)
9. 最後の管理者保護 (ロール変更 / 離脱対象が、当該プロジェクトの最後の admin 行になる場合)—— 違反 → 403 (E-AUTHZ-LAST-ADMIN-PROTECTED)。判定はオーナー由来 admin 行を contract_owners JOIN または NOT IN サブクエリで除外し、メンバー由来の admin 行のみで件数判定する
10. 再認証判定 (重要操作の場合、FR-005)—— 5 分以内 + 1 回限り、未保持なら 403
11. inquiry 境界判定 (end_user の場合)—— 再入室トークンが紐づく inquiry_id のみアクセス可

12. 契約別利用上限判定 (owner_quota_overrides)—— 上限到達 → 429 + Retry-After

1.6.3 6.3 critical 通知の宛先解決 (共有概念正本)

critical 重要度の通知 = メール強制送信。送信先は「**project_users.role='admin' AND valid=1**を1件でも保持するユーザー」(オーナー自身もプロジェクト作成時に admin 行を自動取得するため、本クエリ単独でオーナー + プロジェクト管理者を網羅できる)。

```
SELECT DISTINCT u.id, u.email_encrypted, u.email_hmac, u.email_key_version
FROM users u
JOIN project_users pu ON pu.user_id = u.id
WHERE pu.contract_owner_user_id = :contractOwnerUserId
  AND pu.role = 'admin' AND pu.valid = 1
  AND u.status = 'active' AND u.valid = 1;
```

クエリ設計: オーナー自身が admin 行を持つため、admin 行の JOIN 条件単独でオーナー + プロジェクト管理者を網羅できる。DISTINCT でメンバーが複数プロジェクトで admin を持つ場合の二重カウントを防ぐ。オーナー側も project_users 行が1件以上あるため DISTINCT 対象に含まれる(二重カウント無し)。

部分インデックス idx_project_users_role_admin(WHERE role = 'admin' AND valid = 1) で高速化。詳細は 06_メッセージ一覧.md §2.4 も参照。

1.6.4 6.4 重要操作の再認証 (FR-005)

重要操作	関連画面	API
パスワード変更	(グローバル)	POST /auth/password/change
退会申請	SCR-024	POST /withdrawal-requests
課金情報変更	SCR-015	PATCH /billing/payment-method
メンバー招待 / ロール変更 / アカウント論理削除 / 招待再送	SCR-017-M1(プロジェクト WS)	POST /projects/{id}/members, PATCH /projects/{id}/members/{userId}, DELETE /projects/{id}/members/{userId}, POST /members/{id}/resend-invitation
月次予算上限変更	SCR-015	PATCH /billing/monthly-budget-limit
プロジェクト削除(論理削除)	SCR-026 のみ	DELETE /projects/{id}
ウィジェット ローテーション	SCR-014	POST /projects/{id}/widget-key/rotate
FAQ 削除	SCR-012	DELETE /faqs/{id}

有効期間: 当該操作 1 回のみかつ 15 分以内

1.6.5 6.5 オーナー保護 (FR-333)/ 自己操作不可 (FR-334)/ 単一オーナー所属 (FR-335)

ルール	内容
オーナー保護	オーナー (contract_owners 行を持つユーザー) に対する停止・削除・降格・オーナー解除操作はすべて 403。1 契約 = 1 オーナー (contract_owners.user_id PRIMARY KEY で DB レベル担保)
自己操作不可	自分自身の利用状態変更・削除・ロール降格・プロジェクト割当変更は API 403。特に自分が admin を持つ最後のプロジェクトで自身を member 化 / 割当解除する操作は不可 (E-AUTHZ-SELF-MUTATION)
単一オーナー所属	同一メールでも別オーナー配下は別アカウントとして扱う

1.7 7. アカウント状態

状態	説明	ログイン可否	操作制限
有効	contract_owners.contract_status='active'(所属契約)+ ユーザー自身が users.status='active'	○	なし
招待中	users.status='pending_activation'(招待トークン送信済、未受諾)	×	招待トークンで初回パスワード設定のみ

状態	説明	ログイン可否	操作制限
メール未確認	users.status='pending_verification'	△(SCR-023のみ)	メール確認のみ
停止中(契約)	contract_owners.contract_status='suspended'	○(オーナーのみ)	課金画面 / 退会画面 / データエクスポート画面のみ
ロック中	ログイン失敗 5 回	×(15 分間)	-
退会申請中(契約)	contract_owners.contract_status='deleted_pending'	△(オーナーのみ)	データエクスポート + 退会キャンセル
退会済み(契約)	contract_owners.contract_status='deleted'	×	-

1.7.1 7.1 サスペンション中の認可ゲート (§6.2.7 から移管)

contract_owners.contract_status='suspended' 中:

操作	認可
課金画面 (SCR-015)	許可
データエクスポート画面	許可
退会画面 (SCR-024)	許可
その他	403
ウィジェット応答	423(FR-138)

reason 別のセッション扱い:

- operator_manual / tos_violation: 5 秒以内に全セッション無効化 (FR-022)
- payment_failure_grace_expired / trial_expired_no_payment_method: 既存セッション継続 (課金 / エクスポート / 退会のみ)

解除条件: 顧客管理側の連携 IF #1 受信 (支払い成功 / 運営者解除) で contract_owners.contract_status='active' へ。

1.7.2 7.2 退会・解約フロー (§6.2.8 から移管)

オーナーが SCR-024 で退会申請

- 再認証 FR-005
- contract_owners.contract_status='deleted_pending'
- ウィジェット応答停止、新規 FAQ 登録不可
- 当月末まで利用可
- 月末締めで利用停止 (月割りなし)
- 30 日エクスポート猶予開始
- 物理削除バッチ実行: contract_owners.contract_status='deleted' + valid=0(CHECK で同時セット強制)
- 全データ削除 + announcement_recipients 削除

詳細は 11_課金請求設計書.md §5 参照。

1.7.3 7.3 規約改定フロー (§6.2.9 から移管)

運営者が新規約を準備

- 発効 30 日前: inbox(announcement/critical)+ メール + ステータスページ
- 契約単位で段階的に発効 (全契約同日強制ではない)
- 新規約発効
- ログイン時に同意状況チェック
 - └ Yes (同意済み): 通常画面へ
 - └ No: SCR-025 規約再同意割込み
 - └ 同意期限 14 日以内
 - └ Yes (同意): 通常画面へ
 - └ No (不同意): 退会フロー (SCR-024) へ誘導
 - └ 同意期限超過時: 機能制限ガード (課金画面操作と新規プロジェクト作成は不可、FAQ 編集とウィジェット)

観点	仕様
通知タイミング	改定発効日の 30 日前
通知経路	inbox(announcement/critical)+ メール(強制送信)+ ステータスページ
同意期限	発効日 + 14 日(計 44 日の余裕)
同意期限超過時	SCR-025 強制割込み。ログイン自体は可能、機能制限
段階的实施	契約単位で発効日を分散
非同意時	退会フロー (SCR-024) へ誘導

1.8 8. 認証・認可エラー

詳細は [06_エラー設計書.md § 4.2-4.3](#) を正本とする。本書では主要エラーのみ。

エラー ID	発生条件	メッセージ ID	ステータスコード	システム動作
E-AUTH-CREDENTIAL	パスワード不一致	MSG-SCR-001-ERROR-001	401	失敗カウント +1。攻撃者にヒントを与えない
E-AUTH-LOCKED	5 回失敗	MSG-SCR-001-ERROR-003	423	15 分ロック。本人 + オーナー + 全プロジェクト管理者へ通知
E-AUTH-TOKEN-EXPIRED	トークン期限切れ	MSG-SCR-003-ERROR-001	410	再送 CTA を返却
E-AUTHZ-FORBIDDEN	プロジェクトロール不足	MSG-COMMON-AUTHZ-403	403	統一文言
E-AUTHZ-OWNER-BOUNDARY	オーナー境界違反	MSG-COMMON-NOT-FOUND	404 偽装	リソース存在を漏らさない
E-AUTHZ-PROJECT-DENIED	プロジェクト割当不足	MSG-COMMON-NOT-FOUND	404 偽装	同上
E-BIZ-CONTRACT-SUSPENDED	契約停止中	MSG-BILL-CONTRACT-SUSPENDED	403	課金 / 退会 / エクスポートのみ許可
E-AUTHZ-OWNER-ONLY	オーナー専有機能をメンバーが操作	MSG-COMMON-AUTHZ-403	403	UI 非表示
E-AUTHZ-REAUTH-REQUIRED	再認証期限切れ	MSG-COMMON-REAUTH	403	再認証モーダル表示
E-AUTHZ-OWNER-PROTECTED	オーナー停止 / 削除 / 降格	MSG-COMMON-OWNER-PROTECTED	403	(FR-333)
E-AUTHZ-SELF-MUTATION	自分自身の操作禁止	MSG-COMMON-SELF-MUTATION	403	(FR-334)
E-AUTHZ-TERMS	規約未同意	MSG-SCR-025-*	403 → SCR-025 ヘリダイレクト	規約再同意フロー

1.9 9. 未決事項

No	内容	確認先	期限	ステータス
1	利用者側 admin への MFA	基本設計書 §10.1.5 で MVP 範囲外宣言	Future	既決 (MVP リスク受容、Future 移行)—— 04_将来拡張/01_将来要件定義書.md 参照

1.9.1 9.1 MVP リスク受容 (§10.1.5)

リスク	MVP の補完策	残留リスクレベル
admin 認証情報漏洩(フィッシング/クレデンシャルスタッフィング)	(a) FR-007 ロックアウト (5 回失敗 × 15 分)、(b) FR-177 Turnstile 発動、(c) NFR-505 ログイン通知メール、(d) FR-005 重要操作前再認証、(e) FR-332 アクティブセッション一覧での異常検知	中 (MFA 導入で「低」へ低減可能)
退会・データエクスポート等の不可逆操作の不正実行	(a) FR-005 再認証必須、(b) NFR-505 操作通知メール、(c) 監査ログ (audit_logs、NFR-602a) による事後追跡	中 (MFA 導入で「低」へ)
課金情報変更の不正実行	(a) FR-005 再認証必須、(b) Stripe Elements による PCI スコープ縮小、(c) NFR-505 課金操作通知メール、(d) サスペンション解除には支払成功 Webhook 必須 (§ 6.2.7)	中 (MFA + Stripe 3DS で「低」へ)
orphan admin による単一障害点	FR-333 オーナー保護、FR-334 自己操作不可	低

運用方針: 利用者側 admin はパスワード認証で運用。リスク受容は本節および NFR-330 系セキュリティ管理で文書化。